

UniApp【萌宠案例】下拉刷新与触底加载更多实现

在萌宠案例的分类页中，萌宠列表数据通常较多，下拉刷新和触底加载更多是提升用户体验的核心功能。下拉刷新用于实时更新列表数据，触底加载更多则实现分页加载，避免一次性加载大量数据导致页面卡顿。本方案基于UniApp原生组件与API，结合分页逻辑、状态管理及异常处理，实现流畅稳定的列表交互效果，同时兼容多端运行。

一、核心技术选型与原理

1. 核心组件与API

- scroll-view组件**：作为列表容器，支持纵向滚动，通过`refresher-enabled`开启下拉刷新，`@scrolltolower`监听触底事件，是实现功能的核心容器。
- uni.stopPullDownRefresh()**：下拉刷新完成后，用于关闭刷新动画，必在刷新逻辑的finally中调用。
- 分页参数**：采用`page`（当前页码）和`pageSize`（每页条数）作为分页核心参数，与后端接口配合实现数据分批返回。

2. 核心逻辑设计

- 状态管理**：定义刷新状态（isRefreshing）、加载更多状态（isLoadingMore）、无更多数据标记（noMoreData）及错误信息（errorMsg），避免重复请求与状态混乱。
- 下拉刷新流程**：触发刷新→重置页码为1→请求最新数据→覆盖列表数据→关闭刷新动画→更新无更多数据状态。
- 触底加载流程**：监听触底→判断是否可加载（非刷新中、非加载中、有更多数据）→页码+1→请求下一页数据→追加列表数据→更新无更多数据状态→重置加载状态。
- 异常处理**：刷新/加载失败时，下拉刷新需提示错误，触底加载需回滚页码，确保分页逻辑正常。

二、全局基础配置

1. 分页常量配置（utils/constant.js）

统一管理分页相关常量，便于后续维护与修改。

```

1  /**
2   * 全局常量配置
3   * 包含分页、接口状态等通用常量
4   */
5  export const Constant = {
6    // 分页配置
7    PAGE: {
8      DEFAULT_PAGE: 1, // 默认页码
9      DEFAULT_PAGE_SIZE: 10, // 默认每页条数
10     MAX_PAGE_SIZE: 20 // 最大每页条数（防止请求压力）
11   },
12   // 接口状态码
13   API_CODE: {
14     SUCCESS: 0, // 成功状态码
15     NO_DATA: 1001 // 无数据状态码
16   }
17 }

```

2. 强化请求工具类（utils/request.js）

基于之前的请求工具类，确保分页请求能正常接收数据并处理基础异常。

Code block

```

1  import { asyncUtil } from './asyncUtil'
2
3  const baseConfig = {
4    baseUrl: 'https://api.pet.com',
5    timeout: 10000,
6    headers: {
7      'Content-Type': 'application/json',
8      'access-key': uni.getStorageSync('pet_access_key') || ''
9    }
10 }
11
12 const request = (options) => {
13   const requestOptions = {
14     url: baseConfig.baseUrl + (options.url || ''),
15     method: options.method || 'GET',
16     header: { ...baseConfig.headers, ...options.header },
17     data: options.data || {},
18     timeout: options.timeout || baseConfig.timeout
19   }
20
21   return asyncUtil.promisify(uni.request, requestOptions)
22     .then((res) => {
23       const { statusCode, data } = res

```

```

24     if (statusCode < 200 || statusCode >= 300) {
25       throw new Error(`HTTP错误: ${statusCode}`)
26     }
27     if (data.code !== 0) {
28       throw new Error(`业务错误: ${data.msg || '请求失败'}`)
29     }
30     return data.data
31   })
32   .catch((err) => {
33     const errorInfo = asyncUtil.classifyError(err)
34     console.error(`[请求异常]`, errorInfo.detail)
35     throw errorInfo
36   })
37 }
38
39 request.get = (url, data = {}, options = {}) => request({ url, method: 'GET',
data, ...options })
40 request.post = (url, data = {}, options = {}) => request({ url, method:
'POST', data, ...options })
41
42 export default request

```

三、核心页面实现：分类页下拉刷新与触底加载

以萌宠分类页为载体，完整实现下拉刷新和触底加载更多功能，包含页面结构、逻辑处理与样式适配。

1. 页面结构 (pages/category/category.vue)

使用scroll-view作为列表容器，配置下拉刷新相关属性，添加加载状态提示与错误提示。

Code block

```

1  <template>
2    <view class="category-page">
3
4      <view class="category-tabs">
5        <view
6          class="tab-item"
7          v-for="(tab, index) in categoryTabs"
8          :key="tab.id"
9          :class="{ active: currentTab === index }"
10         @tap="switchTab(index)"
11        >
12          {{ tab.name }}
13        </view>

```

```
14     </view>
15
16
17     <scroll-view
18         class="pet-list-container"
19         scroll-y
20         refresher-enabled
21         refresher-threshold="45"
22         refresher-default-style="black"
23         refresher-background="#f5f5f5"
24         @refresherrefresh="onPullRefresh"
25         @scrolltolower="onReachBottom"
26     >
27
28     <view class="pet-list">
29         <navigator
30             class="pet-item pet-card"
31             v-for="(pet, index) in petList"
32             :key="pet.id + index"
33             :url="'/pages/detail/detail?id=' + pet.id"
34         >
35             <image
36                 :src="pet.coverImg"
37                 mode="widthFix"
38                 class="pet-img"
39                 lazy-load
40                 placeholder-image="/static/placeholder.png"
41                 @error="handleImgError($event)"
42             ></image>
43             <view class="pet-info">
44                 <view class="pet-name">{{ pet.name }}</view>
45                 <view class="pet-breed">品种: {{ pet.breed }}</view>
46                 <view class="pet-meta">
47                     <view class="like-count">
48                         <image src="/static/icon/like.png" mode="widthFix" class="like-
icon"></image>
49                         {{ pet.likeCount }}
50                     </view>
51                     <view class="view-count">浏览 {{ pet.viewCount }}</view>
52                 </view>
53             </navigator>
54         </view>
55
56
57     <view class="status-tip" v-if="petList.length === 0 && !isRefreshing
&& !isLoadingMore">
58         {{ errorMsg || '暂无萌宠数据, 换个分类试试吧~' }}
```

```

59         </view>
60         <view class="loading-tip" v-if="isLoadingMore">加载中...</view>
61         <view class="loading-tip" v-if="noMoreData && petList.length > 0">已加载
全部数据</view>
62     </view>
63 </scroll-view>
64 </view>
65 </template>

```

2. 逻辑实现 (pages/category/category.vue)

核心逻辑包含分类切换、下拉刷新、触底加载、数据请求与异常处理，严格管理各状态避免冲突。

Code block

```

1  <script setup>
2  import { ref, onLoad } from 'vue'
3  import request from '../../utils/request'
4  import { Constant } from '../../utils/constant'
5  import { imageUtil } from '../../utils/imageUtil'
6
7  // 分类数据
8  const categoryTabs = ref([
9    { id: 1, name: '全部', type: 'all' },
10   { id: 2, name: '猫咪', type: 'cat' },
11   { id: 3, name: '狗狗', type: 'dog' },
12   { id: 4, name: '其他', type: 'other' }
13 ])
14 const currentTab = ref(0) // 当前选中分类
15
16 // 列表与分页状态
17 const petList = ref([])
18 const page = ref(Constant.PAGE.DEFAULT_PAGE)
19 const pageSize = ref(Constant.PAGE.DEFAULT_PAGE_SIZE)
20 const isRefreshing = ref(false) // 下拉刷新状态
21 const isLoadingMore = ref(false) // 触底加载状态
22 const noMoreData = ref(false) // 无更多数据
23 const errorMsg = ref('') // 错误信息
24
25 // 页面加载：初始化默认分类数据
26 onLoad(() => {
27   loadPetData()
28 })
29
30 // 切换分类
31 const switchTab = (index) => {
32   if (currentTab.value === index) return

```

```
33     currentTab.value = index
34     // 切换分类时重置状态并加载数据
35     resetListStatus()
36     loadPetData()
37 }
38
39 // 重置列表状态（分类切换/下拉刷新时调用）
40 const resetListStatus = () => {
41     page.value = Constant.PAGE.DEFAULT_PAGE
42     petList.value = []
43     noMoreData.value = false
44     errorMsg.value = ''
45 }
46
47 // 核心：加载萌宠数据
48 const loadPetData = () => {
49     // 获取当前分类类型
50     const currentType = categoryTabs.value[currentTab.value].type
51     const params = {
52         type: currentType,
53         page: page.value,
54         pageSize: pageSize.value
55     }
56
57     request.get('/category/pet', params)
58         .then((res) => {
59             // 严格校验数据格式
60             if (!Array.isArray(res.list)) {
61                 throw new Error('萌宠列表数据格式异常')
62             }
63             const newPetList = res.list.filter(item => item.id && item.coverImg &&
item.name) // 过滤无效数据
64             // 下拉刷新（页码1）：覆盖数据；触底加载：追加数据
65             if (page.value === Constant.PAGE.DEFAULT_PAGE) {
66                 petList.value = newPetList
67             } else {
68                 petList.value = [...petList.value, ...newPetList]
69             }
70             // 判断是否无更多数据（返回数据长度小于每页条数）
71             noMoreData.value = newPetList.length < pageSize.value
72         })
73         .catch((errorInfo) => {
74             // 触底加载失败时回滚页码，避免下次加载跳过
75             if (page.value > Constant.PAGE.DEFAULT_PAGE) {
76                 page.value -= 1
77             }
78             errorMsg.value = errorInfo.msg
```

```

79      // 权限错误特殊处理：引导开启权限
80      if (errorInfo.type === 'permission') {
81        uni.showModal({
82          title: '权限提示',
83          content: errorInfo.msg,
84          confirmText: '去设置',
85          success: (modalRes) => {
86            if (modalRes.confirm) {
87              uni.openSetting()
88            }
89          }
90        })
91      }
92    }
93  }
94  // 权限错误特殊处理：引导开启权限
95  if (errorInfo.type === 'permission') {
96    uni.showModal({
97      title: '权限提示',
98      content: errorInfo.msg,
99      confirmText: '去设置',
100     success: (modalRes) => {
101       if (modalRes.confirm) {
102         uni.openSetting()
103       }
104     }
105   })
106 }

```

3. 样式实现 (pages/category/category.vue)

适配列表布局、状态提示与交互反馈，确保多端显示一致。

Code block

```

1  .category-page {
2    background-color: #f5f5f5;
3    min-height: 100vh;
4  }
5
6  /* 分类标签栏 */
7  .category-tabs {
8    display: flex;
9    justify-content: space-around;
10   align-items: center;
11   height: 88rpx;
12   background-color: #fff;
13   border-bottom: 1rpx solid #eee;
14 }
15 .tab-item {
16   font-size: 28rpx;
17   color: #666;
18   padding: 0 10rpx;
19   height: 100%;
20   display: flex;
21   align-items: center;
22   position: relative;
23 }
24 .tab-item.active {
25   color: #ff6700;
26   font-weight: 500;
27 }
28 .tab-item.active::after {
29   content: '';
30   position: absolute;
31   bottom: 0;

```

```
32     left: 0;
33     width: 100%;
34     height: 6rpx;
35     background-color: #ff6700;
36     border-radius: 3rpx;
37 }
38
39 /* 列表容器 */
40 .pet-list-container {
41     width: 100%;
42     height: calc(100vh - 88rpx);
43     padding: 24rpx;
44     box-sizing: border-box;
45 }
46
47 /* 萌宠列表 */
48 .pet-list {
49     display: flex;
50     flex-wrap: wrap;
51     justify-content: space-between;
52     gap: 24rpx;
53 }
54 .pet-item {
55     width: calc(50% - 12rpx);
56     background-color: #fff;
57     border-radius: 16rpx;
58     overflow: hidden;
59     box-shadow: 0 2rpx 8rpx rgba(0, 0, 0, 0.05);
60 }
61 .pet-img {
62     width: 100%;
63     height: 280rpx;
64 }
65 .pet-info {
66     padding: 20rpx;
67 }
68 .pet-name {
69     font-size: 28rpx;
70     font-weight: 500;
71     margin-bottom: 8rpx;
72 }
73 .pet-breed {
74     font-size: 24rpx;
75     color: #666;
76     margin-bottom: 16rpx;
77 }
78 .pet-meta {
```

```
79     display: flex;
80     justify-content: space-between;
81     font-size: 22rpx;
82     color: #999;
83 }
84 .like-icon {
85     width: 24rpx;
86     height: 24rpx;
87     vertical-align: middle;
88     margin-right: 4rpx;
```

四、多端适配与性能优化

1. 多端适配要点

- **小程序端：**

scroll-view的`scroll-y`属性需配合固定高度使用，否则可能导致滚动异常，本方案通过`calc(100vh - 88rpx)`计算高度。

- 下拉刷新的`refresher-threshold`建议设置为45rpx左右，符合小程序用户操作习惯。

App端：

Android端需注意scroll-view的滚动流畅度，避免嵌套过多复杂组件，可通过`enable-flex`属性优化。

iOS端下拉刷新的背景色与小程序保持一致，提升视觉统一性。

H5端：

H5端scroll-view的滚动可能受浏览器默认样式影响，需在全局样式中清除`body`的`margin`和`padding`。

低版本浏览器不支持`calc`计算，可通过js动态设置scroll-view高度适配。

2. 性能优化技巧

- **状态锁防重复请求：**通过`isRefreshing`和`isLoadingMore`状态锁，防止下拉刷新与触底加载同时触发，或快速触底导致的重复请求。
- **图片懒加载：**列表图片启用`lazy-load`属性，配合`placeholder-image`，减少首屏加载压力，提升页面渲染速度。
- **数据过滤与校验：**请求成功后过滤无效数据（无id、无图片的条目），避免渲染异常，同时减轻页面负担。
- **分页参数限制：**设置`MAX\_PAGE\_SIZE`限制最大每页条数，防止一次性请求大量数据导致的内存占用过高。
- **防抖处理（可选）：**若分类切换频繁，可给`switchTab`方法添加防抖（如300ms），避免短时间内多次请求。

五、常见问题与解决方案

问题现象	原因分析	解决方案
下拉刷新后动画不关闭	未调用`uni.stopPullDownRefresh()`，或调用位置错误（未在finally中）	在`loadPetData`的finally中统一调用`uni.stopPullDownRefresh()`，确保无论成败都执行
触底加载多次重复请求	未添加状态锁，快速滑动到底部时多次触发`scrolltolower`事件	添加`isLoadingMore`状态锁，请求发起时置为true，finally中置为false，锁为true时阻止新请求
切换分类后数据叠加	分类切换时未重置`page`页码与`petList`列表数据	封装`resetListStatus`方法，切换分类时调用，重置页码、列表、无更多数据标记
加载更多后页码混乱	加载失败时未回滚页码，导致下次加载页码错误	在catch回调中判断`page.value > 1`，若满足则`page.value -= 1`回滚页码
scroll-view无法滚动	未设置固定高度，或子元素高度未超过容器高度	给scroll-view设置明确高度（如通过calc计算），确保子元素高度超过容器高度

六、扩展场景：下拉刷新自定义样式

若需要更个性化的下拉刷新效果，可通过`refresher-custom-style`自定义刷新区域样式，配合slot实现自定义加载动画。

Code block

```
1 <scroll-view
2   class="pet-list-container"
3   scroll-y
4   refresher-enabled
5   refresher-threshold="60"
6   :refresher-custom-style="{ backgroundColor: '#f5f5f5', color: '#ff6700' }"
7   @refresherrefresh="onPullRefresh"
8   @scrolltolower="onReachBottom"
9 >
10
11 <template #refresher>
12   <view class="custom-refresher">
```

```

13     <image
14         src="/static/icon/refresh.png"
15         mode="widthFix"
16         class="refresh-icon"
17         :style="{ transform: `rotate(${refreshAngle}deg)` }"
18     ></image>
19     <view class="refresh-text">{{ refreshText }}</view>
20 </template>
21
22 </scroll-view>

```

Code block

```

1  // 自定义刷新相关状态
2  const refreshAngle = ref(0)
3  const refreshText = ref('下拉刷新')
4  const refreshTimer = ref(null)
5
6  // 监听下拉刷新状态变化 (可选)
7  uni.onPullDownRefresh((res) => {
8      // 模拟刷新动画旋转
9      refreshTimer.value = setInterval(() => {
10         refreshAngle.value += 30
11         if (refreshAngle.value >= 360) refreshAngle.value = 0
12     }, 100)
13     refreshText.value = '正在刷新...'
14 })
15
16 // 刷新完成后清除定时器
17 const clearRefreshTimer = () => {
18     if (refreshTimer.value) {
19         clearInterval(refreshTimer.value)
20         refreshTimer.value = null
21     }
22     refreshAngle.value = 0
23     refreshText.value = '下拉刷新'
24 }
25
26 // 在loadPetData的finally中调用
27 .finally(() => {
28     uni.stopPullDownRefresh()
29     clearRefreshTimer()
30     // 其他状态重置
31 })

```

七、总结

本方案基于UniApp原生能力，为萌宠案例分类页实现了完整的下拉刷新与触底加载更多功能，核心优势如下：

1. **状态可控**：通过多状态管理避免重复请求、页码混乱等问题，确保交互流程稳定。
2. **用户体验优**：包含加载提示、无数据提示、错误提示，下拉刷新与加载更多过渡自然，提升用户感知。
3. **多端兼容**：适配小程序、App、H5端，通过样式与逻辑优化确保各端表现一致。
4. **性能可靠**：结合图片懒加载、数据过滤、状态锁等优化手段，降低页面资源占用，提升流畅度。
5. **易于扩展**：支持自定义下拉刷新样式，可根据业务需求快速调整分页参数与交互效果。

该实现方案可直接复用至萌宠案例的其他列表页面（如推荐页、搜索结果页），只需修改请求接口与参数，即可快速实现相同的下拉刷新与触底加载更多功能。

（注：文档部分内容可能由 AI 生成）